

Projet 9

Améliorez l'organisation Salesforce du client Fasha

Livrable n°3

Pipeline d'intégration et de déploiement continu

Parcours Développeur Salesforce - OpenClassrooms

Abderrahim Yachkour

Développeur Salesforce junior - Cabinet SFQUAL

Client : Fasha (distributeur de vêtements)

-

Date de remise : Juin 2026

Table des matières

1. Lien du dépôt et du pipeline	3
2. Pourquoi un pipeline CI/CD	3
3. Comportement du pipeline	3
3.1. Sur Pull Request vers main	3
3.2. Sur push vers main.....	4
4. Composants techniques du pipeline	4
5. Comment configurer le pipeline sur une nouvelle org.....	4
5.1. Générer la paire de clés RSA.....	4
5.2. Créer la Connected App côté Salesforce	4
5.3. Configurer les secrets GitHub	5
6. Comment vérifier que le pipeline fonctionne	5
7. Bénéfices observés	7

1. Lien du dépôt et du pipeline

Le pipeline d'intégration continue est défini dans le fichier `main_deploy.yml` du dossier `.github/workflows` du dépôt GitHub.

URL du dépôt :

<https://github.com/AbderrahimYachkour/P9-Ameliez-organisation-Salesforce-pour-votre-entreprise.git>

URL du workflow :

https://github.com/AbderrahimYachkour/P9-Ameliez-organisation-Salesforce-pour-votre-entreprise/blob/main/.github/workflows/main_deploy.yml

Onglet Actions (exécutions du pipeline) :

<https://github.com/AbderrahimYachkour/P9-Ameliez-organisation-Salesforce-pour-votre-entreprise/actions>

2. Pourquoi un pipeline CI/CD

Avant la mise en place du pipeline, chaque déploiement vers l'organisation Salesforce de Fasha se faisait manuellement, depuis le poste d'un développeur. Cette pratique posait plusieurs problèmes :

- Aucune garantie que les tests Apex sont exécutés avant le déploiement.
- Le déploiement dépend de la configuration locale du développeur (version du Salesforce CLI, variables d'environnement, etc.).
- Aucune trace centralisée des déploiements (qui a poussé quoi, à quelle heure, avec quel résultat).

Le pipeline GitHub Actions résout ces trois problèmes en automatisant la validation et le déploiement, et en conservant l'historique complet des exécutions.

3. Comportement du pipeline

Le pipeline réagit à deux types d'événements Git, avec un comportement différent pour chacun.

3.1. Sur Pull Request vers main

Quand un développeur ouvre une Pull Request qui propose de fusionner une branche dans main, le pipeline lance une validation en mode dry-run. Concrètement :

- Il calcule le delta de métadonnées entre la base (main) et la PR (HEAD).
- Il génère un `package.xml` minimal contenant uniquement les fichiers modifiés.
- Il appelle `sf project deploy validate` avec l'option `RunLocalTests` : Salesforce simule le déploiement, lance tous les tests Apex de l'org, mais n'écrit rien.

- Si la validation échoue (test cassé, métadonnée invalide), la PR est marquée en échec et ne peut pas être fusionnée.

3.2. Sur push vers main

Quand une PR est fusionnée et que le commit arrive sur main, le pipeline lance le déploiement réel :

- Il refait le calcul du delta entre le commit précédent et le nouveau commit.
- Il appelle sf project deploy start (et non plus validate) : cette fois, les modifications sont effectivement écrites dans l'org de Fasha.
- Les tests Apex sont à nouveau lancés (RunLocalTests), pour la même raison que sur la PR.
- Si une étape échoue, le déploiement est annulé : Salesforce fait un rollback, l'org reste dans son état précédent.

4. Composants techniques du pipeline

- @salesforce/cli : outil officiel en ligne de commande pour interagir avec Salesforce (login, deploy, run tests).
- sfdx-git-delta : utilitaire qui calcule le package.xml minimal entre deux commits Git. Cela évite de redéployer toute l'org à chaque modification.
- Authentification JWT : le robot GitHub Actions s'authentifie à Salesforce avec une paire de clés RSA, sans jamais utiliser de mot de passe. La clé privée est stockée comme secret GitHub.
- Quatre secrets GitHub : SF_CONSUMER_KEY, SF_USERNAME, SF_JWT_KEY, SF_INSTANCE_URL. Aucune de ces valeurs n'est visible dans le code.

5. Comment configurer le pipeline sur une nouvelle org

Pour la traçabilité (et au cas où il faudrait régénérer les clés), voici les étapes complètes que j'ai suivies pour mettre en place l'authentification JWT entre GitHub Actions et Salesforce.

5.1. Générer la paire de clés RSA

Depuis un terminal local, j'ai généré une clé privée et un certificat public auto-signé :

- openssl genrsa -out server.key 2048
- openssl req -new -key server.key -out server.csr
- openssl x509 -req -sha256 -days 3650 -in server.csr -signkey server.key -out server.crt
- Le fichier server.crt est uploadé dans la Connected App côté Salesforce.
- Le contenu de server.key est encodé en base64, puis copié dans le secret GitHub SF_JWT_KEY (le workflow le décode au moment de l'authentification).

5.2. Créer la Connected App côté Salesforce

Dans l'organisation Salesforce, en tant qu'administrateur :

- Setup, App Manager, New Connected App.
- Renseigner le nom (par exemple GitHub Actions Deploy) et l'email de contact.
- Activer Enable OAuth Settings.
- Activer Use digital signatures et uploader le fichier server.crt généré à l'étape précédente.
- Sélectionner les scopes OAuth : Manage user data via APIs (api), Perform requests at any time (refresh_token, offline_access).
- Sauvegarder. Patienter environ dix minutes (le temps que Salesforce propage la Connected App).
- Récupérer la Consumer Key affichée sur la page de la Connected App : c'est elle qui ira dans le secret SF_CONSUMER_KEY.

5.3. Configurer les secrets GitHub

Depuis l'interface du dépôt GitHub :

- Settings, Secrets and variables, Actions, New repository secret.
- Créer SF_CONSUMER_KEY avec la valeur récupérée à l'étape précédente.
- Créer SF_USERNAME avec le nom d'utilisateur Salesforce (typiquement une adresse email).
- Créer SF_JWT_KEY en collant le contenu du fichier server.key encodé en base64 (commande `base64 -w 0 server.key`, puis copier le résultat).
- Créer SF_INSTANCE_URL avec l'URL de l'org (par exemple `https://login.salesforce.com` pour une org de production, `https://test.salesforce.com` pour une sandbox).

6. Comment vérifier que le pipeline fonctionne

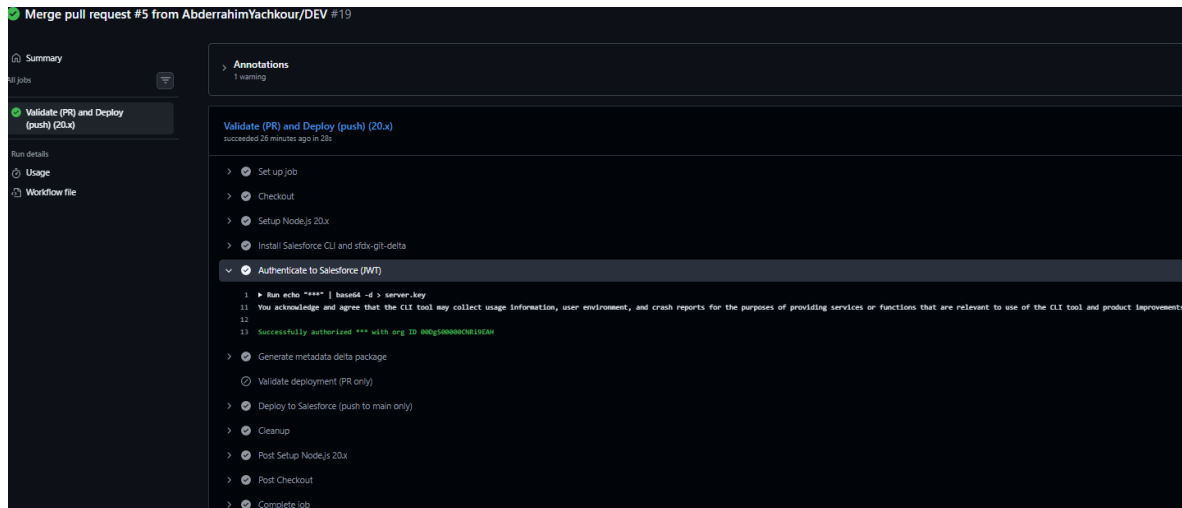
Une fois la configuration en place, il suffit de pousser un commit sur une branche feature et d'ouvrir une PR vers main pour voir le pipeline démarrer. Trois endroits permettent de suivre une exécution :

- Onglet Actions du dépôt GitHub : liste de tous les runs avec leur statut.
- Page d'une PR ouverte : un check (succès ou échec) apparaît en bas avec le lien vers le log.
- Log d'un run : permet de voir chaque étape, son temps d'exécution et sa sortie standard.

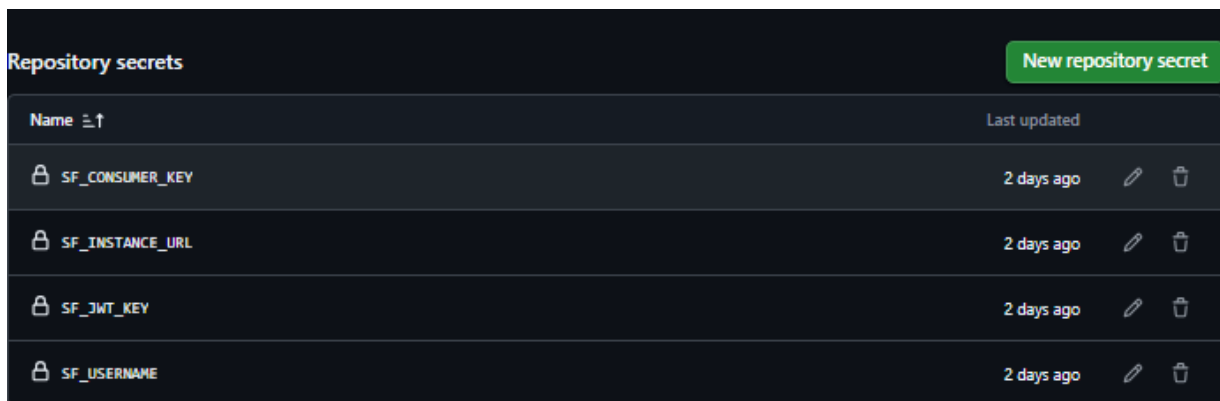
Onglet Actions du dépôt GitHub

Workflow	Event	Status	Branch	Actor
Merge pull request #5 from AbderrahimYachkour/DEV	Push	Success	DEV	AbderrahimYachkour
ci: précise que SF_JWT_KEY doit être encodé en base64	Pull request	Success	DEV	AbderrahimYachkour
Merge pull request #4 from AbderrahimYachkour/DEV	Push	Success	DEV	AbderrahimYachkour
fix: Correction delta CI	Pull request	Success	DEV	AbderrahimYachkour
Merge pull request #3 from AbderrahimYachkour/DEV	Push	Success	DEV	AbderrahimYachkour
release: corrections triggers, tests, CI et exposition LWC orders	Pull request	Success	DEV	AbderrahimYachkour

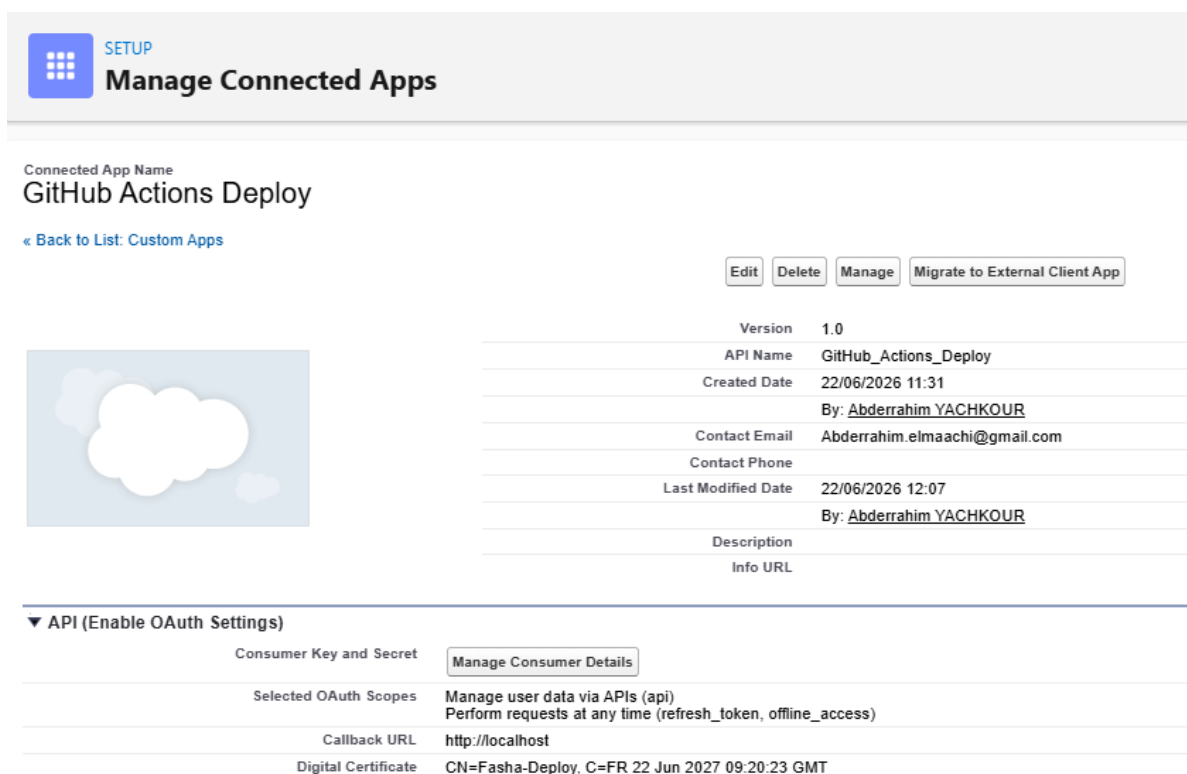
Détail d'un run réussi



Secrets GitHub configurés



Connected App côté Salesforce



7. Bénéfices observés

- Aucun déploiement vers Fasha ne peut désormais avoir lieu sans avoir passé tous les tests Apex de l'org.
- Les déploiements sont nettement plus rapides qu'un déploiement complet, car seules les métadonnées modifiées sont transmises.
- L'historique complet des déploiements est conservé dans l'onglet Actions, avec qui a poussé quoi, quand et avec quel résultat.
- Aucun mot de passe n'est échangé ni stocké : l'authentification JWT élimine ce risque de sécurité.
- Le pipeline étant défini dans un fichier versionné, sa propre évolution est tracée comme n'importe quel changement de code.